Plan To Resolve Under-Specified AutoLISP / Visual LISP Semantics

Codex

June 24, 2026

Contents

1 Purpose

This document defines a concrete plan to resolve the currently under-specified areas in ‘autolisp-spec/documentation/autolisp-visual-lisp-specification-draft.org‘.

The goal is to promote uncertain sections from:

- under-specified
- inferred

to:

- documented
- or tested and version-qualified

2 Core Principle

Not all evidence sources have the same authority.

For language-specification work, the evidence ranking should be:

1. Official vendor documentation.
2. Executable tests against real AutoCAD and BricsCAD releases.
3. Official vendor examples and shipped sample code.
4. Large corpora of real-world AutoLISP code.
5. Third-party implementations.
6. Community discussions and folklore.

This means:

- code corpus scanning is useful,
- third-party implementations are useful,
- but neither should override vendor docs or real-product tests.

3 Short Answer to the Open Questions

3.1 Would scanning libraries of AutoLISP code help?

Yes, but mainly as secondary evidence.

It is useful for:

- discovering syntax forms actually used in the wild,
- estimating which edge cases matter in practice,
- finding idiomatic use of special forms,
- locating undocumented but commonly accepted patterns,
- building a compatibility test corpus.

It is not sufficient for:

- proving that a syntax form is valid,
- determining exact reader acceptance rules,
- resolving version-specific behavior,
- distinguishing vendor bugs from language semantics.

3.2 Would reading sources of other implementations help?

Yes, but also as secondary evidence.

It is useful for:

- finding plausible semantic interpretations,
- discovering tricky implementation areas,
- identifying test cases already encountered by others,
- comparing differing design assumptions.

It is not sufficient for:

- establishing normative behavior,
- deciding between Autodesk and BricsCAD divergence,
- resolving undocumented historical behavior without tests.

4 Execution Tracking

Actionable tasks for this subproject now live in ‘autolisp-spec/PLAN.md‘.

This document remains the place for evidence policy, resolution method, and specification-context notes.

5 Open Questions and Best Resolution Method

5.1 Exact Reader Grammar

Best method:

1. vendor docs,
2. executable tests,
3. corpus scan,
4. implementation comparison.

Reason:

- this is where vendor docs are weak and product tests are strongest.

5.2 Numeric Literal Syntax

Best method:

1. product tests,
2. vendor docs,
3. corpus scan,
4. third-party implementations.

Reason:

- this is exactly the kind of topic where documentation is often too informal.

5.3 String Escape Repertoire

Best method:

1. vendor docs,
2. product tests,
3. corpus scan.

5.4 Special Form Semantics

Best method:

1. dedicated vendor pages,
2. product tests,
3. corpus scan.

5.5 Error Behavior

Best method:

1. vendor docs,
2. product tests,
3. real-world error-handling code,
4. implementation comparison.

5.6 Version Drift

Best method:

1. vendor release notes,
2. versioned product tests.

6 Current Status

Phase 5 closure landed on 2026-04-26: the five long-running open items from this document — ‘atoi‘, ‘atof‘, ‘vl-member-if-not‘, ‘exit‘ vs ‘quit‘, and the printer surface (‘prin1‘ / ‘princ‘ / ‘print‘ / ‘terpri‘ / ‘prompt‘) — were all closed to **documented** using the BricsCAD V26 per-symbol pages reached via Claude-in-Chrome MCP.

Each entry now carries:

- Authoritative AutoCAD + BricsCAD per-symbol citations.
- Tested-when-product-available probe pointers (probe sources are already in ‘autolisp-spec/sources/‘; ‘scripts/run-probes.sh‘ will execute them under any AutoCAD / BricsCAD reachable through ‘$\text{RUNNER}_{\text{TEMPLATE}}$‘).
- An **Implementation-defined Behaviour** section on ‘atoi‘ / ‘atof‘ documenting the lex edges that are still vendor-prose-silent.

No entries remain at ‘summary-documented‘ or ‘family-documented‘ status. The Phase-2 follow-up stubs (377 BricsCAD-only entries added 2026-04-26 after the live-TOC walk) are out of scope for **this** document — they are tracked separately in ‘vendor-inventory-2026.org‘ §8.13 and ‘PLAN.md‘.

7 Resolved Questions (2026-04-26)

7.1 ‘atoi‘

Closed via direct citation of the BricsCAD V26 ‘atoi‘ page, which is more explicit than Autodesk’s reference. Documented behaviour:

- Trailing-junk strictness: parsing stops at the first non-numeric character; ‘(atoi ”12.34”)‘ returns ‘12‘, ‘(atoi ”xyz”)‘ returns ‘0‘.
- Decimal-fraction input is truncated toward zero.
- The lexical edges below remain **implementation-defined** and are recorded as such in the spec entry (no vendor prose pins them down):
  - leading whitespace handling,
  - sign acceptance (‘+17‘, ‘-17‘),
  - leading-zero forms (‘017‘ — decimal vs octal-like),
  - ‘0x‘-prefixed forms,

  - trailing-junk strictness for partial numbers.

- `clautolisp` implementation-defined choices are recorded in the spec entry.
- The probe suite in ‘sources/probe-atoi.lsp‘ will record AutoCAD vs BricsCAD product behaviour when run via ‘scripts/run-probes.sh‘ against a live product.

7.2 ‘atof‘

Closed via direct citation of the BricsCAD V26 ‘atof‘ page. Documented behaviour: same trailing-junk-stops-parsing model as ‘atoi‘; ‘(atof ”12.34”)‘ returns ‘12.34‘; ‘(atof ”xyz”)‘ returns ‘0.0‘.

The lexical edges below remain **implementation-defined** (recorded as such in the spec entry):

- ‘.5‘, ‘1.‘ decimal-point variants,
- exponent notation,
- leading whitespace,
- locale (decimal comma vs decimal point),
- hexadecimal floating syntax,
- trailing-junk strictness.

‘clautolisp‘ implementation-defined choices are recorded in the spec entry. Probe suite in ‘sources/probe-atof.lsp‘.

7.3 ‘vl-member-if-not‘

Closed via direct citation of the BricsCAD V26 dedicated per-symbol page. The page provides the per-element control wording that Autodesk’s family-table entry only summarises:

- Validation of each list item stops at the first item where the comparator returns ‘nil‘.
- The remainder of the list, from that element onward, is returned.
- Returns ‘nil‘ when no element matches the (negated) condition.

The Phase-3 inferred-symmetry-with-‘vl-member-if‘ hypothesis is now confirmed by direct citation.

7.4 Printer / Display Surface

Closed via direct citation of the BricsCAD V26 ‘prin1‘, ‘princ‘, ‘print‘, ‘terpri‘, and ‘prompt‘ per-symbol pages. The previously open distinctions are now explicit:

- ‘prin1‘ rewrites control characters (‘^\” ‘) so output round-trips through the reader.
- ‘princ‘ writes control characters verbatim.
- ‘print‘ is ‘prin1‘ with a leading newline and a trailing space.
- ‘terpri‘ is zero-arity, command-line-only, returns ‘nil‘.
- ‘prompt‘ returns ‘nil‘ and is the recommended channel for end-user command-line output (Bricsys explicitly recommends ‘prompt‘ over the printer functions for messages).
- ‘(prin1)‘ and ‘(princ)‘ with no arguments return the **void** symbol value, conventionally used to terminate a ‘defun‘ without echoing a result.

7.5 ‘exit‘ vs ‘quit‘

Closed via direct citation of the BricsCAD V26 ‘exit‘ and ‘quit‘ per-symbol pages. Both functions:

- Are zero-arity.
- Unconditionally abort the running LISP code.
- Print ‘; error : quit / exit abort‘ to the prompt as the abort propagates.

Bricsys distinguishes the verbs only in prose (”exits” vs ”cancels”); the **signature, message, and effect** are documented identically. The historical hypothesis that some vendor or release distinguished ‘exit‘ and ‘quit‘ is not supported by current vendor evidence. Treat as alternative spellings of the same primitive.

8 Future Validation Work

Phase 5 closed all five long-running open items at the **documented** level using direct vendor citations. As of 2026-04-26, four of the five items have **also** been product-tested against BricsCAD V26 on macOS (atoi, atof, terpri, the printer surface). Tested-status notes are recorded in each spec entry's "Tested Behaviour" sub-section and reference `results/bricscad/macos/20260426T122808Z/results`.

The remaining work is:

- Run 'scripts/run-probes.sh autocad' against a real AutoCAD installation when a license seat becomes available, using the dedicated direct-AutoCAD runner committed at 'scripts/run-probes-direct-autocad.sh'. The runner was authored and tested up to AutoCAD splash on 2026-04-26 against AutoCAD 2026 / macOS, but the local install had no licensed seat available and the probe was aborted. AutoCAD product-test parity is the only open Phase-5 follow-up.
- Re-run the probes against future BricsCAD releases when they ship, to maintain the version-qualified evidence trail.

9 Probe-Suite Findings (BricsCAD V26 / macOS, 2026-04-26)

The probe run captured the following normative findings (see `results.sexp`):

- **atoi** — strtol-style: skips leading whitespace, accepts optional sign, parses the longest decimal-digit prefix, returns 0 on no digits. '0x'-prefixed forms return 0 (no auto-detection). '017' is parsed as decimal, not octal. '3.9' truncates to 3.
- **atof** — strtod-style. Notably, '(atof "0x1p4")' returns '16.0', i.e. BricsCAD V26 **does** accept C99 hexadecimal-floating syntax. Decimal-comma is **rejected** (no locale sensitivity on the macOS build): '(atof "3,5")' returns '3.0'.
- **terpri** — zero-arity. The 2-argument form '(terpri stream)' raises `too few / too many arguments at [TERPRI]`.
- **prin1** / **princ** / **print** — all accept the 2-argument form '(fn expr stream)'. Verbatim representations confirmed: 'prin1' writes the round-trippable form (with quotes for strings), 'princ' writes the value verbatim, 'print' writes " + prin1 + " ".

- **prompt** — returns ‘nil‘; output goes to the command-line channel only and is not capturable through a file handle.

A side-finding that extended Phase 4: BricsCAD V26 ‘or‘ returns **T or nil**, not the first non-nil value. This was discovered by running probe-core against a fresh BricsCAD instance — the original ‘(setq x (or x default))‘ idiom in probe-core silently clobbered ‘x‘ with ‘T‘. probe-core has been patched to use ‘(if (not x) (setq x default))‘, and the spec’s ‘or‘ entry now documents the divergence with a portable-idiom note.

A second side-finding: the ‘autolisp-script‘ wrapper at ‘~/works/sncf-reseau/src/outils-autolisp/autolisp-script/‘ redefines ‘prin1‘, ‘princ‘, ‘print‘, and ‘prompt‘ to single-argument helpers that mirror command-line output. The 2-argument forms must therefore be tested via the direct ‘bricscad -B run.scr‘ runner (‘scripts/run-probes-direct.sh‘), not via the wrapper.

10 Final Recommendation

Phase 5 followed the resolution-order recommendation:

1. Official vendor documentation (BricsCAD V26 per-symbol pages reached via Claude-in-Chrome MCP).
2. Real product tests — pending a local CAD installation.
3. Corpus scanning — out of scope for Phase 5.
4. Third-party implementation study — out of scope.

The closure model the spec now uses for every entry that came through Phase 5 is exactly the one this document recommended: each unresolved question has been turned into either a documented citation, a deferred-but-scripted product test, or an explicitly marked implementation-defined choice.